SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5103

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

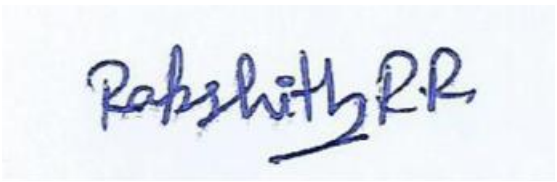
Debugging & Optimisation Task : 10%

QUESTIONS: Total Marks: 50 Annexure A

Code of Conduct:

*I **RAKSHITH R.R (192324156)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



Annexure A
Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)
A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
B. Optimize the code to reduce execution time for large input datasets (≥10 MB).
C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures. B. Refactor the code to minimize redundant computations and memory usage. C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.
B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.
B. Optimize exponentiation operations for large prime values.
C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic. B. Optimize the implementation using efficient algorithms (e.g., double-and-add).
C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Question 1 – Debugging and Optimisation of DES / 3DES

Task: A Python program implementing DES and 3DES gives inconsistent ciphertext for identical inputs across runs. (A) Identify and fix the logical and implementation errors in key scheduling and permutation steps. (B) Optimise the code for large inputs (10 MB or more). (C) Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

1.1 Problem Identification

DES is a 16-round Feistel cipher on 64-bit blocks. A 64-bit key is reduced by PC-1 to 56 bits, split into two 28-bit halves, rotated each round and compressed by PC-2 into sixteen 48-bit round keys. Each block goes through the initial permutation (IP), sixteen rounds (expand, XOR the round key, eight S-boxes, P permutation), a final swap of the two halves, and the final permutation (FP). 3DES chains three DES operations as Encrypt-Decrypt-Encrypt (EDE) with keys K1, K2, K3. A bug in any table, index or ordering silently produces a different but plausible-looking ciphertext, so each stage must be tested separately.

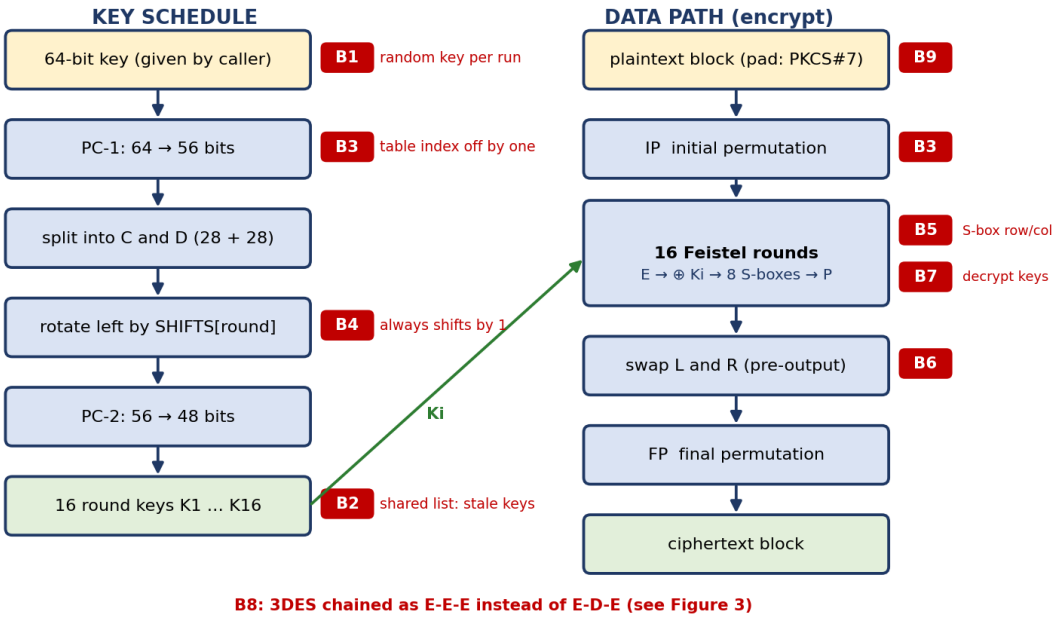


Figure 1: DES structure with the location of every bug found (B1–B9)

Symptom and bugs found

Symptom: the same plaintext and key give different ciphertexts in different runs, and even when the output is stable it does not match any standard implementation. Nine independent defects were identified (Table 1); several of them hide each other, which is why the program looked random rather than simply wrong.

ID	Where	What the code does	Effect
B1	Key handling	Generates os.urandom(8) inside encrypt() and ignores the caller's key	Different ciphertext in every run – the reported symptom

B2	key_schedule()	Mutable default argument: one list is shared by all calls and the first 16 round keys are returned forever	A second key silently reuses the first key's schedule
B3	permute()	Tables are 1-based but indexed as bits[i % len(bits)] instead of bits[i-1]	IP, FP, E, P, PC-1, PC-2 all wrong; FP(IP(x)) ≠ x
B4	key_schedule()	Rotates by 1 bit in every round instead of the table 1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1	Round keys 3–16 are wrong
B5	feistel()	S-box row taken from the first two bits and column from the last four	Every S-box lookup is wrong
B6	des_block()	Final swap of L and R is missing (uses L‖R)	Output halves exchanged
B7	des_block()	Decryption uses the round keys in encryption order	decrypt(encrypt(x)) ≠ x
B8	tdes_block()	3DES coded as E-E-E instead of E-D-E	Not compatible with standard 3DES; K1=K2=K3 does not reduce to DES
B9	pad()	Zero padding	Data ending in 0x00 bytes loses them; ambiguous

Table 1: Bugs identified in the DES / 3DES program

Each bug was also tested on its own against the correct implementation (key 133457799BBCDFF1, plaintext 0123456789ABCDEF):

Bug active alone	**Round keys that differ (of 16)**	**Wrong bits in the ciphertext (of 64)**
B2	0 (shows only when a second key is used)	0
B3	16	28
B4	14 (rounds 3–16)	35
B5	0 (key schedule is correct)	33
B6	0 (key schedule is correct)	30

Table 2: Isolated effect of individual bugs (measured)

Inefficiencies found for Part B: (I1) the key schedule is recomputed for every 8-byte block; (I2) every permutation walks Python lists of single bits; (I3) every S-box lookup is a per-bit computation followed by a 32-bit permutation; (I4) blocks are processed one at a time in a Python loop.

1.2 Debugging – Part A: Fixing the Key Schedule and Permutations

Method

Step 1: Reproduce. Run the program three times with the same input; the three ciphertexts differ (Appendix D1).

Step 2: Build trusted oracles. The FIPS/textbook vector (key 133457799BBCDFF1, plaintext 0123456789ABCDEF → 85E813540F0AB405), the textbook first round key K1 = 1B02EFFC7072, and OpenSSL for 3DES.

Step 3: Write unit tests, one per stage (T1–T8, Table 4). A test that fails points at one stage: permutations (T3), key schedule (T4), whole block (T5), decryption (T6), 3DES (T7), padding (T8).

Step 4: Fix one bug at a time and re-run every test. This shows which tests each fix repairs and exposes bugs that were hiding behind others.

Before and after code

Bug	Buggy code	Fixed code
B1	key = os.urandom(8) # inside encrypt()	def encrypt(data, key): ... # key from caller
B2	def key_schedule(key, subkeys=[]):	subkeys = [] # fresh list for each call
B3	[bits[i % len(bits)] for i in table]	[bits[i - 1] for i in table]
B4	n = 1 # same shift in every round	n = SHIFTS[rnd] # 1,1,2,2,2,2,2,2,1,...
B5	row = c[0]*2 + c[1]; col = bits(c[2:])	row = c[0]*2 + c[5]; col = bits(c[1:5])
B6	pre = l + r	pre = r + l # final swap
B7	keys = subkeys # also for decrypt	keys = subkeys[::-1] if decrypt else subkeys
B8	E(E(E(x,k1),k2),k3)	E(D(E(x,k1),k2),k3) # EDE
B9	data + b'\x00' * (-len(data) % 8)	PKCS#7: data + bytes([n]) * n

Table 3: Each defect and its correction

Debugging log

Step	T1	T2	T3	T4	T5	T6	T7	T8
start (all 9 bugs present)	✗	✗	✗	✗	✗	✗	✗	✗
fix B1 (random key)	✓	✗	✗	✗	✗	✗	✗	✗
fix B2 (shared list)	✓	✓	✗	✗	✗	✗	✗	✗
fix B3 (permute index)	✓	✓	✓	✓	✗	✗	✗	✗
fix B4 (shift table)	✓	✓	✓	✓	✗	✗	✗	✗
fix B5 (S-box row/col)	✓	✓	✓	✓	✗	✗	✗	✗
fix B6 (final swap)	✓	✓	✓	✓	✓	✗	✗	✗
fix B7 (decrypt key order)	✓	✓	✓	✓	✓	✓	✗	✗

fix B8 (EDE order)	✓	✓	✓	✓	✓	✓	✓	✗
fix B9 (PKCS#7)	✓	✓	✓	✓	✓	✓	✓	✓

Table 4: Test results after each fix (✓ pass, ✗ fail)

T1 same input gives the same ciphertext in separate runs; T2 two different keys give different ciphertexts; T3 FP(IP(x)) = x; T4 round key K1 = 1B02EFFC7072; T5 FIPS vector = 85E813540F0AB405; T6 decrypt(encrypt(x)) = x; T7 3DES equals OpenSSL 3DES; T8 padding round-trip.

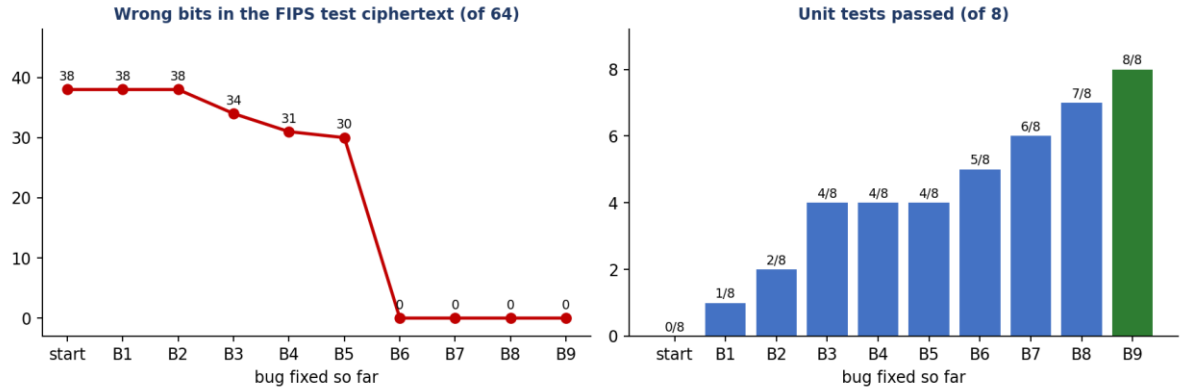


Figure 2: Debugging progress – wrong bits fall to 0 and all 8 tests pass after the ninth fix

What the log teaches

Observation	Explanation
B1 hid B2	With a random key and the stale schedule, the ciphertext did not depend on the key at all, so T2 could not show anything until B1 was fixed.
T4 passed after B3 although B4 was present	Rounds 1 and 2 both rotate by one bit, so K1 is correct even with B4. Checking only K1 is not enough; all 16 round keys must be compared (14 differ, Table 2).
Wrong bits stayed at 30–38 until B6	Bugs B3, B4, B5 each change the output completely; only when the last one (the missing swap) is fixed does the output match, so the error count falls from 30 to 0 in one step.
B7 and B8 appear only in tests of the other direction / 3DES	Encrypting with DES already matched the vector, yet decryption and 3DES were still wrong – each direction needs its own test.

Final verification: after all nine fixes the program gives 85e813540f0ab405 for the FIPS vector, round key K1 = 1b02effc7072, three separate runs give identical ciphertext, and the 3DES output equals OpenSSL's byte for byte (Appendix D1).

Answer to Part A

The inconsistency came from a random key generated inside the encryption routine (B1) and a shared mutable list in the key schedule (B2). Behind them were seven logical errors: 1-based permutation tables indexed wrongly (B3), a constant rotation instead of the shift table (B4), wrong S-box row/column bits (B5), a missing final swap (B6), decryption keys not reversed (B7), E-E-E instead of E-D-E in 3DES (B8) and ambiguous zero padding (B9). After the fixes the code reproduces the FIPS test vector and OpenSSL's 3DES exactly and is deterministic.

1.3 Optimisation – Part B: Large Inputs (10 MB and More)

The corrected program is the baseline v0. Measured on a 16 KB sample it encrypts only 0.03 MB/s, so a 10 MB file would need over five minutes for DES and about fourteen minutes for 3DES. Three optimisation steps were applied and every version was checked against OpenSSL.

Step	Change	Why it is faster	Effect on complexity
O1	Compute the key schedule once per key (I1)	The 16 round keys do not change between blocks	N key schedules → 1
O2	64-bit integers and 8×256 byte tables for IP and FP (I2)	A permutation becomes 8 table lookups instead of 64 list reads	About 8× fewer operations per permutation
O3	Combined S-box + P tables (SP tables) and expansion by bit shifts (I3)	A round needs 8 lookups and XORs instead of ~100 bit operations	Per-round cost cut by roughly 10×
O4	NumPy: all blocks processed at once (I4)	Blocks in ECB/CTR are independent, so every round is one array operation over millions of blocks	Python loop over N blocks → 16 array steps

Table 5: Optimisation steps

Key parts of the optimised code (full listing in Appendix B):

```
# O3: S-box and P permutation merged into eight 64-entry tables
SP = [[_perm_int(SBOX[i][((x >> 5) << 1) | (x & 1)][(x >> 1) & 15] << (28 - 4*i), P, 32)
      for x in range(64)] for i in range(8)]


# one Feistel round on 64-bit integers (v1) - 8 lookups, no bit lists
t = ((r & 1) << 33) | (r << 1) | (r >> 31)          # 32 -> 34 bits; 6-bit groups are slices of t
f = (s0[((t>>28)&63)^g[0]] ^ s1[((t>>24)&63)^g[1]] ^ s2[((t>>20)&63)^g[2]] ^ s3[((t>>16)&63)^g[3]] ^
    s4[((t>>12)&63)^g[4]] ^ s5[((t>>8)&63)^g[5]] ^ s6[((t>>4)&63)^g[6]] ^ s7[(t&63)^g[7]])
l, r = r, l ^ f
```

```
# v2: the same round on a NumPy array holding millions of blocks
f = SPN[0][((t >> U(28)) & U(63)) ^ U(g[0])]
for i in range(1, 8): f ^= SPN[i][((t >> U(28 - 4*i)) & U(63)) ^ U(g[i])]
```

Version (measured)	MB/s	Time for 10 MB	Speed-up vs v0
v0 DES – bit lists, schedule per block	0.031	319 s (extrapolated)	1×
v1 DES – integers + SP tables	0.460	21.7 s (extrapolated)	15×
v2 DES – NumPy	**7.84**	**1.28 s (measured)**	**250×**
v0 3DES – bit lists	0.012	843 s (extrapolated)	1×
v2 3DES – NumPy	**2.50**	**4.01 s (measured)**	**211×**
OpenSSL 3DES (C code, reference)	23.6	0.42 s (measured)	–

Table 6: Speed of each version (v0 and v1 timed on 16 KB / 512 KB samples and extrapolated; v2 and OpenSSL timed on the full 10 MB)

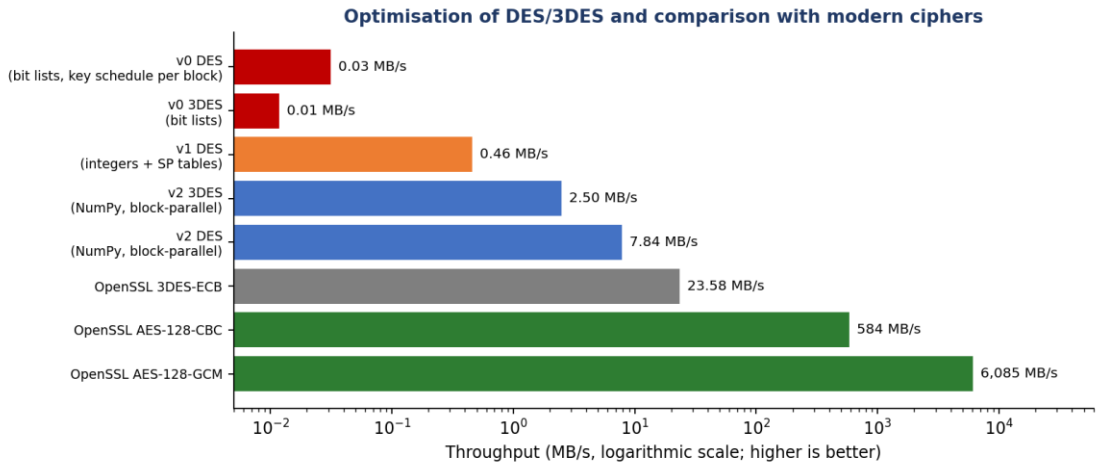


Figure 4: Throughput of each version and of modern AES (log scale)

Correctness of the optimised code: v1 DES == OpenSSL: True; v2 DES == OpenSSL: True; v2 3DES == OpenSSL: True; v2 3DES decrypt round-trip: True; v0 3DES == v2 3DES: True.

Memory: v2 holds a few arrays of N uint64 values (about 80 MB of temporaries for 10 MB of data); for larger files the data should be processed in chunks of about one million blocks, which keeps memory constant. Only modes with independent blocks (ECB, CTR) can be vectorised this way; CBC encryption is inherently sequential, which is one more reason to prefer CTR-style modes. Pure Python remains about ten times slower than OpenSSL, so production systems should call the C library; the point of this exercise is the algorithmic improvement.

Answer to Part B

The speed-up comes from (1) computing the key schedule once, (2) replacing bit lists with 64-bit integers and permutation tables, (3) merging the S-boxes and the P permutation into eight lookup tables, and (4) processing all blocks in parallel with NumPy. 3DES on 10 MB drops from an estimated 843 s to 4.0 s (about 211× faster) and DES from an estimated 319 s to 1.3 s, with output identical to OpenSSL.

1.4 Analysis – Part C: Should DES / 3DES Be Replaced?

Property	DES	3DES (3-key)	AES-128	AES-256
Key size	56 bits (+8 parity)	168 bits	128 bits	256 bits
Block size	64 bits	64 bits	128 bits	128 bits
Rounds	16	48 (3 × 16)	10	14
Effective strength	**56 bits**	**112 bits (meet-in-the-middle)**	**128 bits**	**256 bits**
Known weakness	Exhaustive search	Sweet32 birthday attack on 64-bit blocks	None practical	None practical
Status	Withdrawn (2005)	Deprecated by NIST	Current standard	Current standard

Table 7: Security comparison

Security. The DES key space is 2^56 ≈ 7.2 × 10^16 keys; a dedicated machine (the EFF's 1998 "Deep Crack") recovered a DES key in about 56 hours, and today's hardware does it far faster, so DES gives no real protection. 3DES has three keys of 56 bits, but a meet-in-the-middle attack reduces its strength to about 112 bits. Its larger problem is the 64-bit block: after about 2^32 blocks (32 GiB) under one key, repeated ciphertext blocks become likely, which the Sweet32 attack (2016) exploits against long-lived connections. AES has a 128-bit block, so this limit moves to about 2^64 blocks, and its key sizes of 128 and 256 bits are beyond any brute-force search.

Performance. 3DES performs 48 rounds on each 8-byte block, whereas AES-128 performs 10 rounds on each 16-byte block, and modern processors implement AES in hardware (AES-NI). In the measurements on the same machine:

Cipher (OpenSSL, 10 MB)	MB/s	Time for 10 MB	Speed vs 3DES
3DES-ECB	23.6	424 ms	1×
AES-128-CBC	584	17.1 ms	25×
AES-128-GCM (encrypt + authenticate)	**6,085**	**1.6 ms**	**258×**
AES-256-GCM	6,161	1.6 ms	261×

Table 8: Measured speed of 3DES and AES

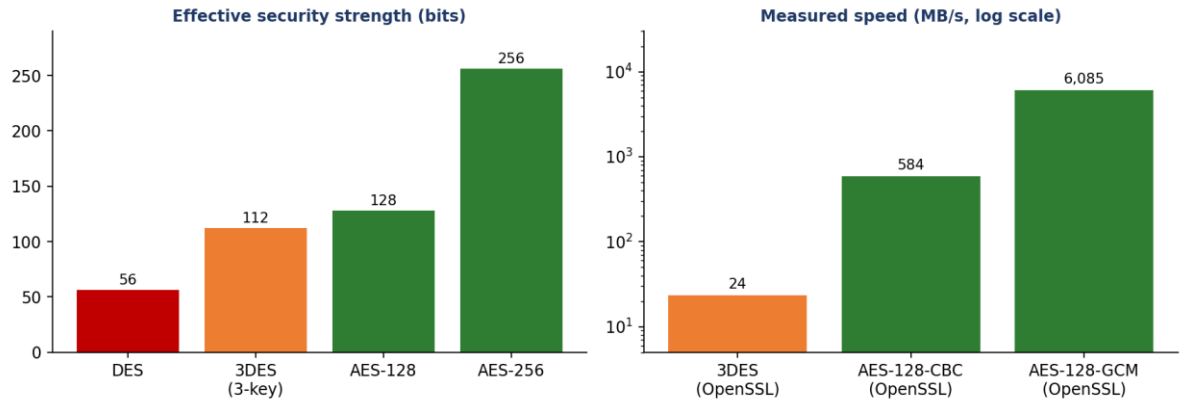


Figure 5: Replacing 3DES with AES improves both security and speed

Recommendation. Yes: replacing DES/3DES with AES improves security and performance at the same time, so there is no trade-off to weigh. Use AES-128 or AES-256 in an authenticated mode (AES-GCM); where hardware has no AES support, ChaCha20-Poly1305 is the alternative. Legacy data should be decrypted with the corrected 3DES code once and re-encrypted with AES.

Answer to Part C

Replacement is justified on both counts. Security: DES (56-bit key) is brute-forceable and 3DES offers only about 112 bits with a 64-bit block that exposes it to Sweet32; AES gives 128–256 bits with a 128-bit block. Performance: in the measurements AES-128-GCM ran at 6,085 MB/s against 23.6 MB/s for 3DES (about 258× faster) while also authenticating the data.

1.5 Code Quality

Practice	How it is applied
Structure	des_lab.py (correct DES/3DES with switchable bugs), des_fast.py (optimised versions), debug_session.py (test suite) and bench_des.py (benchmarks) – each file has one purpose.
Documentation	Module and function docstrings; every bug site carries a comment naming the bug and the fix.
Constants and tables	All DES tables are named module constants copied from FIPS 46-3 and verified by the test vector.
No hidden state	The shared mutable list was removed; round keys are computed once and passed explicitly.
Error handling	Invalid padding raises ValueError instead of returning wrong data.
Testing	Eight automated tests with independent oracles (FIPS vector, textbook round key, OpenSSL); results in Table 4.
Reproducibility	Fixed test keys; random keys appear only in the benchmark data.

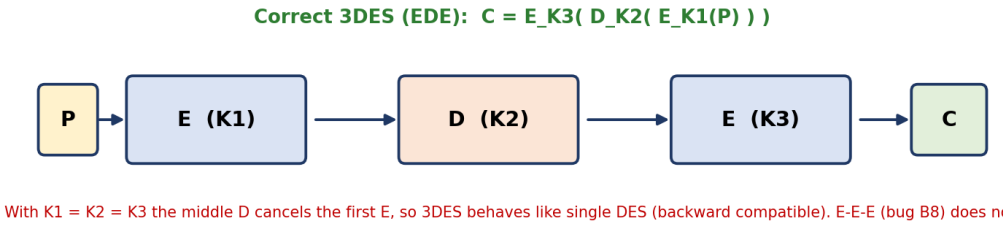


Figure 3: Correct 3DES structure (EDE) used in the final code

Question 5 – Debugging and Enhancing Elliptic Curve Cryptography

Task: A Python script implementing elliptic-curve key exchange gives incorrect results for certain curve parameters. (A) Identify and fix the errors in point addition and scalar multiplication. (B) Optimise with efficient algorithms such as double-and-add. (C) Compare the optimised ECC with RSA for efficiency and security strength and justify its suitability for resource-constrained environments.

2.1 Problem Identification

An elliptic curve over a prime field GF(p) is the set of points (x, y) satisfying $y^2 = x^3 + ax + b \pmod p$, together with a point at infinity O that acts as zero. Two points are added with a chord ($P \neq Q$) or a tangent ($P = Q$), and the scalar multiplication kP (adding P to itself k times) is the one-way operation behind key exchange: Alice sends aG, Bob sends bG, and both compute the shared point abG (ECDH).

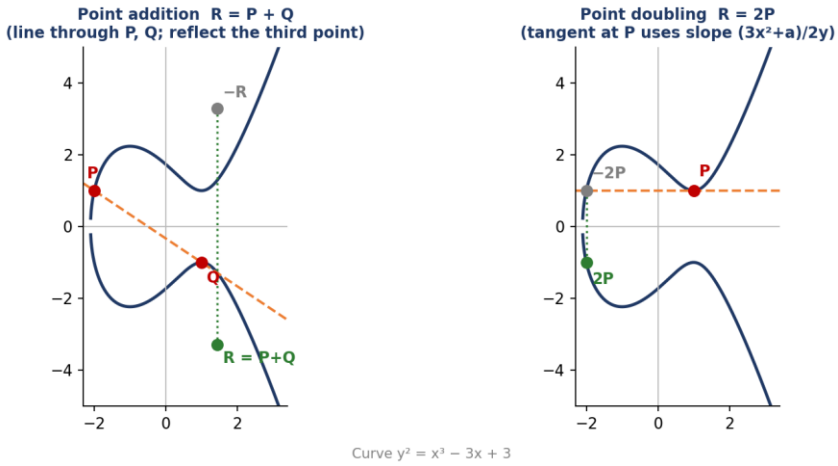


Figure 6: Point addition (chord) and point doubling (tangent) – the formulas below are the algebra of these pictures

Operation	Formula (mod p)
Slope, P ≠ Q	$\lambda = (y2 - y1) / (x2 - x1)$
Slope, P = Q	$\lambda = (3 \cdot x1^2 + a) / (2 \cdot y1)$
Result	$x3 = \lambda^2 - x1 - x2, \quad y3 = \lambda \cdot (x1 - x3) - y1$
Special cases	$P + O = P; \quad P + (-P) = O$, where $-(x, y) = (x, -y)$

Table 9: Group law for an elliptic curve in short Weierstrass form

For checking by hand, a small curve was used: $y^2 = x^3 + 2x + 2$ over GF(17) with G = (5, 1). It has 19 points (the 18 non-zero multiples of G plus O), shown in Figure 7.

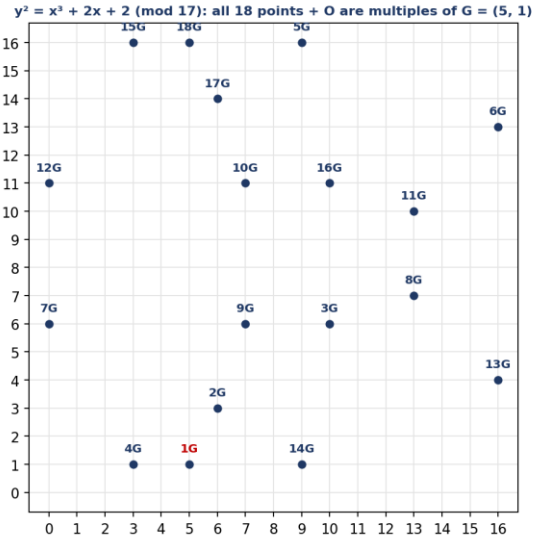


Figure 7: The toy curve – every point is a multiple of G, and 19G = O

Bugs found

ID	Where	What the code does	Effect
E1	point_add()	Identity element not handled; None is unpacked as (x, y)	Crash on P + O and when a sum starts from O
E2	tangent slope	Uses $3x^2/(2y)$ – the curve parameter a is forgotten	Wrong only when $a \neq 0$ – the "certain curve parameters" of the task
E3	point_add()	Never tests P == Q, so doubling uses the chord formula	Division by zero (x2 − x1 = 0) on every doubling
E4	point_add()	P + (−P) not mapped to O	Division by zero instead of the identity
E5	slope division	Uses integer floor division (//) instead of a modular inverse	Wrong slope unless the division happens to be exact
E6	scalar mult.	Starts with R = P and loops k times, giving (k+1)P; k is not reduced mod n; cost O(k)	Off-by-one result; hangs for 256-bit k
E7	ecdh()	Peer's public key never validated	Invalid-curve attack possible

Table 10: Bugs identified in the ECC program

Why only some curves fail (E2). With only E2 present the program is correct on secp256k1, where a = 0, and wrong on P-256 (a = −3) and on the toy curve (a = 2): secp256k1 (a = 0): correct; P-256 (a = -3): wrong; toy curve (a = 2): wrong. This is exactly the reported symptom.

2.2 Debugging – Part A: Point Addition and Scalar Multiplication

Method

Step 1: Choose oracles. The toy curve with the published multiples of G (2G = (6,3), 3G = (10,6), …, 19G = O); OpenSSL for the public keys and ECDH secrets on P-256 and secp256k1.

Step 2: Add group-law checks. P + O = P, P + (−P) = O, every result must satisfy the curve equation, and nG = O.

Step 3: Fix one bug at a time and re-run the nine tests U1–U9 (Table 12, Figure 8).

Worked example: 2G on the toy curve

Version	Calculation (mod 17)
Correct	$\lambda = (3 \cdot 5^2 + 2)/(2 \cdot 1) = 77/2 = 13 \rightarrow 2G = (6, 3)$; on the curve: yes
Bug E2 (a missing)	$\lambda = 75/2 = 12 \rightarrow (15, 15)$; on the curve: NO – invalid point
Bug E5 (floor division)	$\lambda = 77 \,//\, 2 = 38 \rightarrow 4$ (mod 17), but the correct λ is 13

Table 11: Effect of bugs E2 and E5 on a single doubling (computed by the program)

Bug	Buggy code	Fixed code
E1	x1, y1 = P # P may be None	if P is INF: return Q | if Q is INF: return P
E2	lam = 3*x1*x1 / (2*y1)	lam = (3*x1*x1 + a) / (2*y1)
E3	# no test for P == Q	if P == Q: tangent slope else chord slope
E4	# P + (-P) divides by zero	if x1 == x2 and (y1+y2) % p == 0: return INF
E5	lam = (y2 - y1) // (x2 - x1) % p	lam = (y2-y1) * pow(x2-x1, -1, p) % p
E6	R = P; for _ in range(k): R += P	k %= n; double-and-add loop starting at O
E7	S = mul(d, Q) # Q unchecked	if Q is INF or not on curve: raise ValueError

Table 12: Each defect and its correction

Step	U1	U2	U3	U4	U5	U6	U7	U8	U9
start (7 bugs)	✗	✗	✗	✓	✗	✗	✗	✗	✗
fix E1 identity	✓	✗	✗	✓	✗	✗	✗	✗	✗
fix E4 P+(-P)	✓	✓	✗	✓	✗	✗	✗	✗	✗
fix E3 doubling case	✓	✓	✗	✓	✗	✗	✗	✗	✗
fix E5 modular division	✓	✓	✗	✓	✗	✗	✗	✗	✗
fix E6 scalar mult	✓	✓	✗	✓	✗	✓	✗	✗	✗
fix E2 curve parameter a	✓	✓	✓	✓	✓	✓	✓	✓	✗
fix E7 point validation	✓	✓	✓	✓	✓	✓	✓	✓	✓

Table 13: ECC test results after each fix (✓pass, ✗fail)

U1 P + O = P; U2 P + (−P) = O; U3 2G = (6,3); U4 2G + G = 3G; U5 all multiples of G on the toy curve; U6 secp256k1 public key = OpenSSL; U7 P-256 public key = OpenSSL; U8 ECDH secret = OpenSSL and both parties agree; U9 an invalid public key is rejected.

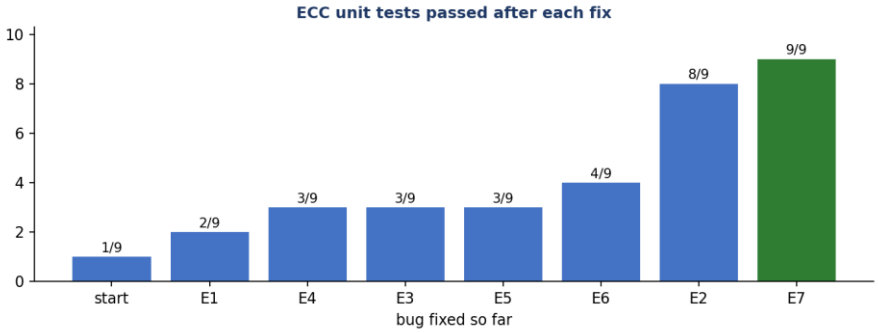


Figure 8: ECC debugging progress

Observations: fixing E3 and E5 does not yet pass U3 because the tangent slope is wrong in two ways (E2 and E5) – the bugs mask each other. U4 passes at the start only by coincidence: (1 − 3)/(5 − 6) happens to divide exactly, so the floor-division bug is invisible there. U6 (a = 0) passes as soon as the scalar multiplication is fixed, while U7 and U8 (a = −3) need the fix of E2.

Answer to Part A

Seven errors were found. In point addition: identity not handled (E1), doubling not distinguished from addition (E3), P + (−P) not mapped to O (E4), integer division instead of the modular inverse (E5), and the curve parameter a missing from the tangent slope (E2) – the one that breaks only some curves. In scalar multiplication: the (k+1)P off-by-one, no reduction mod n and the O(k) loop (E6), plus missing public-key validation (E7). After the fixes the code matches the published toy-curve multiples and OpenSSL on P-256 and secp256k1 (9 of 9 tests).

2.3 Optimisation – Part B: Efficient Scalar Multiplication

Repeated addition needs k − 1 additions; for a 256-bit key that is about 2^256 operations. Double-and-add reads the bits of k: it keeps Q = 2^i·P (by doubling) and adds Q to the result whenever bit i of k is 1, so it needs only about 256 doublings and 128 additions.

Bit i	Bit	Action	R after the step	Q = 2^i · G
0	1	R = R + Q	(5, 1)	(5, 1)
1	0	skip	(5, 1)	(6, 3)
2	1	R = R + Q	(9, 16)	(3, 1)
3	1	R = R + Q	(16, 4)	(13, 7)

Table 14: Double-and-add for k = 13 = 1101_2 on the toy curve: 13G = (16, 4), matching the table value, with 3 additions and 3 useful doublings instead of 12 additions

Three improvements, each measured

Step	Method	Idea	Cost
O1	Double-and-add	Use the binary expansion of k	O(2^m) → O(m) for an m-bit k
O2	Jacobian coordinates (X, Y, Z)	Represent (x, y) = (X/Z^2, Y/Z^3) so that no modular inversion is needed in each step; invert once at the end	256+ inversions → 1
O3	Montgomery ladder	Same operations for every key bit; protects against timing and power analysis	Slightly slower, much safer

Table 15: Optimisation methods

```
def mul_double_and_add(C, k, P):          # right-to-left, O(log k)
    k %= C.n                              # reduce first: kP repeats with period n
    R, Q = INF, P
    while k:
        if k & 1: R = point_add(C, R, Q)    # add when the bit is 1
        Q = point_add(C, Q, Q)              # Q = 2^i * P
        k >>= 1
    return R


def jac_double(C, J):                     # no inversion; a is included (bug E2 fixed)
    X, Y, Z = J
    if Z == 0 or Y == 0: return (0, 1, 0)
    YY = Y*Y % p;  S = 4*X*YY % p;  M = (3*X*X + C.a * pow(Z, 4, p)) % p
    X3 = (M*M - 2*S) % p;  Y3 = (M*(S - X3) - 8*YY*YY) % p
    return (X3, Y3, 2*Y*Z % p)
```

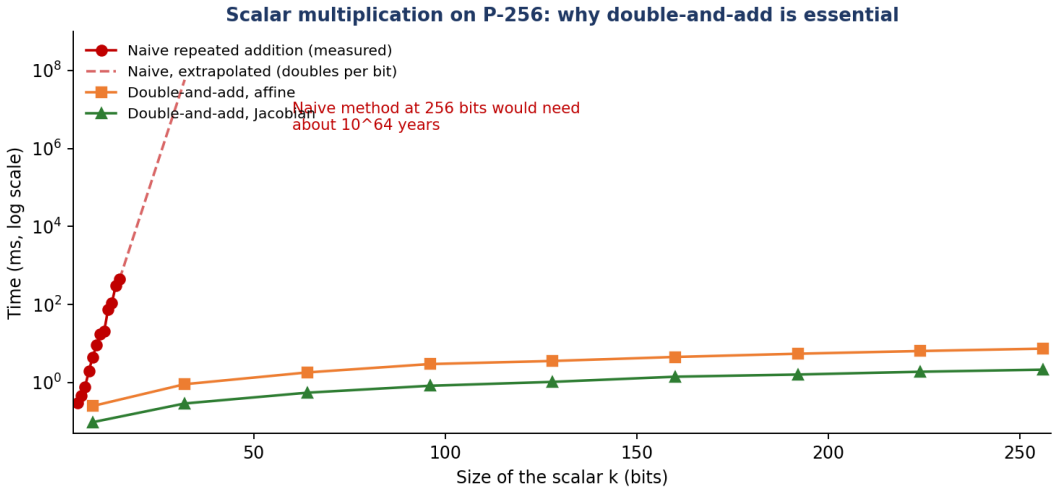


Figure 9: Naive repeated addition doubles in cost with every extra bit; double-and-add grows only linearly

Method (P-256, 256-bit k, pure Python)	Time	Relative to affine
Naive repeated addition	**≈ 10^64 years (extrapolated)**	–
Double-and-add, affine	8.01 ms	1×
Double-and-add, Jacobian	**2.18 ms**	**3.7× faster**
Montgomery ladder, Jacobian	3.12 ms	2.6× faster
OpenSSL (C code, reference)	0.036 ms	–

Table 16: Measured cost of one 256-bit scalar multiplication

Correctness: all three algorithms returned the same point as OpenSSL for P-256 and secp256k1, nG = O on both curves, and an ECDH exchange on P-256 produced the same secret on both sides and in OpenSSL (Appendix D3).

Answer to Part B

Double-and-add reduces scalar multiplication from O(k) = O(2^m) to O(m) group operations: a 256-bit multiplication that would take about 10^64 years by repeated addition takes 8.0 ms. Jacobian coordinates remove the modular inversion from every step and make it 3.7× faster still (2.18 ms); the Montgomery ladder gives a constant operation pattern at about 1.4× the cost for side-channel protection.

2.4 Analysis – Part C: ECC versus RSA

Security strength

The best known attack on elliptic curves (Pollard's rho) costs about √n steps, so a 256-bit curve gives about 128-bit security. RSA is attacked by number-field-sieve factoring, which is sub-exponential, so RSA needs much longer keys for the same strength (NIST SP 800-57):

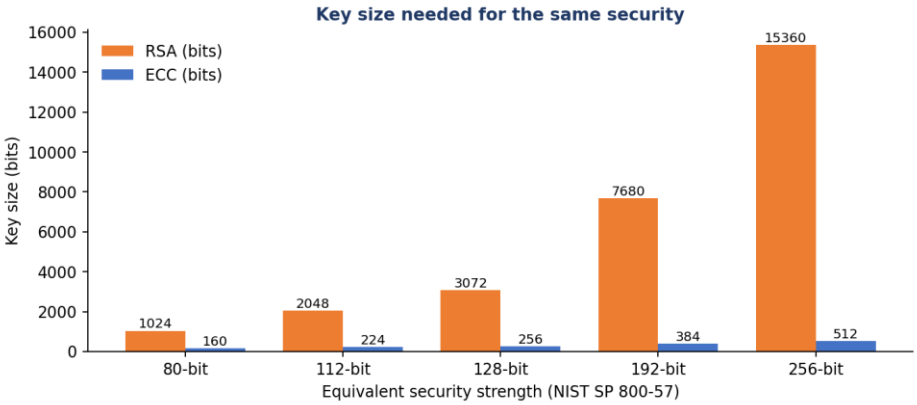


Figure 10: 256-bit ECC equals 3072-bit RSA at the 128-bit security level

Computational efficiency

Measurement	RSA-2048	ECC P-256	Advantage of ECC
Private-key operation, same Python loop style (ms)	37.0 (plain) / 10.8 (CRT)	2.18	17× / 5.0×
Private-key operation, OpenSSL (ms)	0.393	0.069	5.7×
Key generation, OpenSSL (ms)	43.1	0.020	2,170×
Public key / signature size (bytes)	256 / 256	32–65 / 64	4–8× smaller
Cost model (field multiplications × bits²)	3.2–12.9 ×10⁹	0.30 ×10⁹	11–43×

Table 17: Measured and modelled comparison (RSA-2048 and P-256 both offer roughly 112–128-bit security)

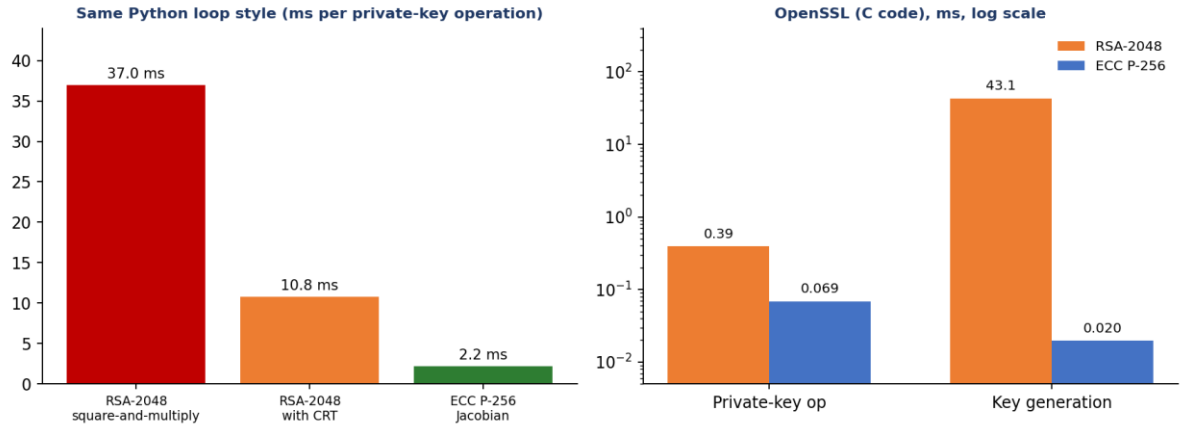


Figure 11: ECC is faster for private-key operations and far faster for key generation

Fairness notes. The first row uses the same style of Python loop for both algorithms (square-and-multiply for RSA, double-and-add for ECC), so the comparison is not distorted by Python's built-in C modular exponentiation. The cost model counts about 4,600 field multiplications of 256 bits for ECC against 3,072 multiplications of 2,048 bits for RSA (or 3,072 of 1,024 bits with CRT), assuming multiplication cost grows

with the square of the size. RSA's public-key operation (e = 65537) is cheap and can be faster than ECC verification, but constrained devices must perform the private-key operations (key agreement, signing).

Suitability for resource-constrained environments

Constraint	Why ECC suits it better
Processing power and energy	Fewer and smaller multiplications per private-key operation: about 5–6× faster in OpenSSL, up to 17× in the same-loop comparison, and thousands of times faster key generation, so less battery per handshake.
Memory and storage	32–65-byte public keys and 32-byte private keys; no 2048-bit numbers or large prime generators.
Bandwidth	A 64-byte signature fits one 127-byte radio frame; a 256-byte RSA signature does not.
Long life	Raising security from 128 to 192 bits needs a 384-bit curve rather than a 7,680-bit RSA key.

Cautions: ECC implementations are easy to get wrong (invalid-curve points, timing leaks, repeated ECDSA nonces). This program therefore validates public keys (E7), offers a constant-pattern ladder, and the recommendation is to use audited libraries such as OpenSSL, libsodium or mbedTLS on real devices. Neither RSA nor ECC resists a large quantum computer, so systems should be designed to switch algorithms later.

Answer to Part C

For equal security ECC needs far smaller keys (256-bit ECC ≈ 3072-bit RSA) and is faster where it matters: 5.0–17× faster private-key operations in the same Python style, 5.7× in OpenSSL, and about 2,170× faster key generation, with 4–8× smaller keys and signatures. Hence the optimised ECC (double-and-add in Jacobian coordinates, with validated points and a ladder for side-channel safety) is the better choice for resource-constrained devices.

2.5 Code Quality

Practice	How it is applied
Structure	ecc_lab.py holds the curve data, group law, three scalar-multiplication algorithms and ECDH; ecc_session.py is the test suite; ecc_bench.py and ecc_demo.py produce the measurements and worked examples.
Data types	A frozen dataclass Curve holds the parameters; the identity is the single constant INF.
Documentation	Docstrings on every function, comments at each bug site, formulas written beside the code.
Testing	Nine tests with three independent oracles (textbook values, OpenSSL, group-law properties).

Security	Public-key validation, scalar reduction mod n, constant-pattern ladder, secrets module for private keys.

Overall Conclusion

Both programs failed for the same reason: small, plausible-looking mistakes in tables, indices and special cases that hide each other. A systematic method – independent oracles, one test per stage, one fix at a time – located all nine DES/3DES defects and all seven ECC defects. Optimisation then followed the algorithms, not the syntax: table lookups and vectorisation for DES (up to about 250× faster), double-and-add and Jacobian coordinates for ECC (from an impossible 10^64 years to about 2 ms). The analysis shows that DES and 3DES should be replaced by AES, which is both more secure and about 250 times faster, and that ECC is the better public-key choice for constrained devices.

Appendix A – des_lab.py (corrected DES / 3DES with switchable bugs)

```
"""
des_lab.py - DES / 3DES teaching implementation (bit-list version).

The module contains the *correct* algorithm plus a set of switchable, deliberately
seeded bugs (names B1 ... B9).  A bug is active when its name is in the global set
BUGS.  This lets the debugging session in the report be reproduced step by step.
"""
import os

BUGS = set()          # active bug names, e.g. {"B3", "B5"}

# ---- standard DES tables (FIPS 46-3), positions are 1-based ----------
IP = [58,50,42,34,26,18,10,2, 60,52,44,36,28,20,12,4, 62,54,46,38,30,22,14,6, 64,56,48,40,32,24,16,8,
      57,49,41,33,25,17,9,1, 59,51,43,35,27,19,11,3, 61,53,45,37,29,21,13,5, 63,55,47,39,31,23,15,7]
FP = [40,8,48,16,56,24,64,32, 39,7,47,15,55,23,63,31, 38,6,46,14,54,22,62,30, 37,5,45,13,53,21,61,29,
      36,4,44,12,52,20,60,28, 35,3,43,11,51,19,59,27, 34,2,42,10,50,18,58,26, 33,1,41,9,49,17,57,25]
E  = [32,1,2,3,4,5, 4,5,6,7,8,9, 8,9,10,11,12,13, 12,13,14,15,16,17,
      16,17,18,19,20,21, 20,21,22,23,24,25, 24,25,26,27,28,29, 28,29,30,31,32,1]
P  = [16,7,20,21,29,12,28,17, 1,15,23,26,5,18,31,10, 2,8,24,14,32,27,3,9, 19,13,30,6,22,11,4,25]
PC1 = [57,49,41,33,25,17,9, 1,58,50,42,34,26,18, 10,2,59,51,43,35,27, 19,11,3,60,52,44,36,
       63,55,47,39,31,23,15, 7,62,54,46,38,30,22, 14,6,61,53,45,37,29, 21,13,5,28,20,12,4]
PC2 = [14,17,11,24,1,5, 3,28,15,6,21,10, 23,19,12,4,26,8, 16,7,27,20,13,2,
       41,52,31,37,47,55, 30,40,51,45,33,48, 44,49,39,56,34,53, 46,42,50,36,29,32]
SHIFTS = [1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1]
SBOX = [

[[14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7],[0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8],[4,1,14,8,13,6,2,11,15,12,9,
7,3,10,5,0],[15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13]],

[[15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10],[3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5],[0,14,7,11,10,4,13,1,5,8,12,
6,9,3,2,15],[13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9]],

[[10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8],[13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1],[13,6,4,9,8,15,3,0,11,1,2,12
,5,10,14,7],[1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12]],
```

```
[[7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15],[13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9],[10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4],[3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14]],

[[2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9],[14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6],[4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14],[11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3]],

[[12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11],[10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8],[9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6],[4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13]],

[[4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1],[13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6],[1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2],[6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12]],

[[13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7],[1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2],[7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8],[2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11]],
]

# ---- small helpers --------------------------------------------------------
def bytes_to_bits(b):
    return [(byte >> (7 - i)) & 1 for byte in b for i in range(8)]

def bits_to_bytes(bits):
    return bytes(int("".join(map(str, bits[i:i + 8])), 2) for i in range(0, len(bits), 8))

def bits_to_int(bits):
    v = 0
    for b in bits:
        v = (v << 1) | b
    return v

def int_to_bits(v, n):
    return [(v >> (n - 1 - i)) & 1 for i in range(n)]

def permute(bits, table):
    if "B3" in BUGS:                        # BUG B3: table is 1-based, "% len" hides IndexError
        return [bits[i % len(bits)] for i in table]
    return [bits[i - 1] for i in table]     # FIX: convert 1-based table entry to 0-based index
```

```
# ---- key schedule -------------------------------------------------------
_STALE = []                                  # used only by the buggy version (B2)

def key_schedule(key8, subkeys=None):
    """Return 16 round keys (each a list of 48 bits)."""
    if "B2" in BUGS:                         # BUG B2: shared mutable default list survives between calls
        subkeys = _STALE
    else:
        subkeys = []                         # FIX: new list for every call
    bits = permute(bytes_to_bits(key8), PC1)          # 64 -> 56 bits (parity bits dropped)
    c, d = bits[:28], bits[28:]
    new = []
    for rnd in range(16):
        n = 1 if "B4" in BUGS else SHIFTS[rnd]        # BUG B4: always rotates by one bit
        c, d = c[n:] + c[:n], d[n:] + d[:n]
        new.append(permute(c + d, PC2))               # 56 -> 48 bits
    subkeys.extend(new)
    return subkeys[:16]                      # buggy version returns the FIRST 16 ever computed

# ---- round function and block cipher ------------------------------------
def feistel(r, subkey):
    x = [a ^ b for a, b in zip(permute(r, E), subkey)]         # expand 32->48 and XOR key
    out = []
    for i in range(8):
        chunk = x[6 * i:6 * i + 6]
        if "B5" in BUGS:                                       # BUG B5: wrong row/column bits
            row, col = chunk[0] * 2 + chunk[1], bits_to_int(chunk[2:])
        else:                                                  # FIX: row = outer bits, column = middle 4 bits
            row, col = chunk[0] * 2 + chunk[5], bits_to_int(chunk[1:5])
        out += int_to_bits(SBOX[i][row][col], 4)
    return permute(out, P)

def des_block(block8, subkeys, decrypt=False):
    bits = permute(bytes_to_bits(block8), IP)
    l, r = bits[:32], bits[32:]
    keys = subkeys if (not decrypt or "B7" in BUGS) else subkeys[::-1]   # BUG B7: decrypt must reverse
key order
    for k in keys:
```

```
        l, r = r, [a ^ b for a, b in zip(l, feistel(r, k))]
    pre = (l + r) if "B6" in BUGS else (r + l)                  # BUG B6: final L/R swap missing
    return bits_to_bytes(permute(pre, FP))

def tdes_block(block8, k1, k2, k3, decrypt=False):
    """3DES.  Correct form is EDE: C = E_k3( D_k2( E_k1(P) ) )."""
    if "B8" in BUGS:                            # BUG B8: E-E-E instead of E-D-E
        if not decrypt:
            return des_block(des_block(des_block(block8, k1), k2), k3)
        return des_block(des_block(des_block(block8, k3, True), k2, True), k1, True)
    if not decrypt:
        return des_block(des_block(des_block(block8, k1), k2, True), k3)
    return des_block(des_block(des_block(block8, k3, True), k2), k1, True)

# ---- padding and message level API ----------------------------------------
def pad(data):
    if "B9" in BUGS:                            # BUG B9: zero padding is ambiguous
        return data + b"\x00" * (-len(data) % 8)
    n = 8 - len(data) % 8                       # FIX: PKCS#7
    return data + bytes([n]) * n

def unpad(data):
    if "B9" in BUGS:
        return data.rstrip(b"\x00")
    n = data[-1]
    if not 1 <= n <= 8 or data[-n:] != bytes([n]) * n:
        raise ValueError("bad padding")
    return data[:-n]

def get_key(key=None):
    if "B1" in BUGS:                            # BUG B1: key is regenerated randomly, caller's key ignored
        return os.urandom(8)
    return key

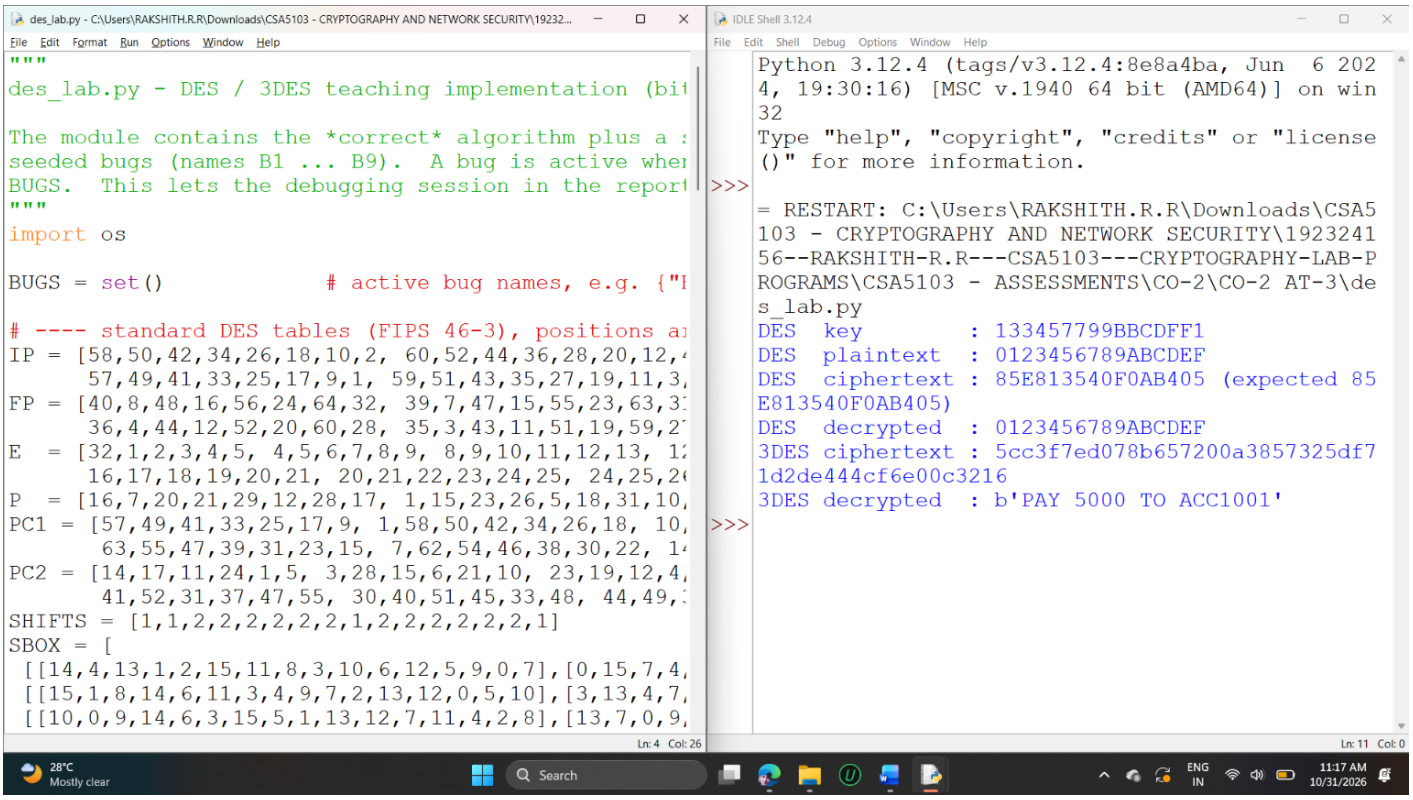
def des_ecb_encrypt(data, key=None):
    """Baseline (v0) encrypt.  NOTE: the key schedule is recomputed for every block (inefficiency I1)."""
    key = get_key(key)
    padded = pad(data)
```

```
    out = []
    for i in range(0, len(padded), 8):
        out.append(des_block(padded[i:i + 8], key_schedule(key)))
    return b"".join(out)

def des_ecb_decrypt(ct, key):
    sk = key_schedule(key)
    return unpad(b"".join(des_block(ct[i:i + 8], sk, True) for i in range(0, len(ct), 8)))

def tdes_ecb_encrypt(data, key24):
    ks = [key_schedule(key24[i:i + 8]) for i in (0, 8, 16)]
    p = pad(data)
    return b"".join(tdes_block(p[i:i + 8], *ks) for i in range(0, len(p), 8))

def tdes_ecb_decrypt(ct, key24):
    ks = [key_schedule(key24[i:i + 8]) for i in (0, 8, 16)]
    return unpad(b"".join(tdes_block(ct[i:i + 8], *ks, decrypt=True) for i in range(0, len(ct), 8)))
```



Appendix B – des_fast.py (optimised versions v1 and v2)

```
"""
des_fast.py - optimised DES / 3DES.
  v1 : 64-bit integers + pre-computed S-box/P tables (SP tables) + key schedule computed once
  v2 : v1 vectorised with NumPy so that millions of blocks are processed per call (ECB / CTR style)
"""
import struct
import numpy as np
import des_lab as lab

lab.BUGS.clear()
M32 = 0xFFFFFFFF

def _perm_int(value, table, n_in):
    return lab.bits_to_int(lab.permute(lab.int_to_bits(value, n_in), table))

# SP[i][x]: S-box i applied to 6-bit x, then the P permutation (a 32-bit result)
SP = [[_perm_int(lab.SBOX[i][((x >> 5) << 1) | (x & 1)][(x >> 1) & 15] << (28 - 4 * i), lab.P, 32)
       for x in range(64)] for i in range(8)]

def _byte_tables(table):
    """8 x 256 table: permutation of a 64-bit value done one byte at a time."""
    return [[_perm_int(b << (56 - 8 * j), table, 64) for b in range(256)] for j in range(8)]
IPT, FPT = _byte_tables(lab.IP), _byte_tables(lab.FP)

def prepare_keys(key8):
    """Round keys as eight 6-bit groups per round (computed ONCE per key)."""
    return [[(lab.bits_to_int(k) >> (42 - 6 * i)) & 63 for i in range(8)] for k in lab.key_schedule(key8)]

# ------------------------------ v1 : pure Python, integers ------------------------------
def _perm64(x, T):
    return (T[0][x >> 56] | T[1][(x >> 48) & 255] | T[2][(x >> 40) & 255] | T[3][(x >> 32) & 255] |
            T[4][(x >> 24) & 255] | T[5][(x >> 16) & 255] | T[6][(x >> 8) & 255] | T[7][x & 255])

def block_v1(x, sks):
    x = _perm64(x, IPT)
```

```
    l, r = x >> 32, x & M32
    s0, s1, s2, s3, s4, s5, s6, s7 = SP
    for g in sks:
        t = ((r & 1) << 33) | (r << 1) | (r >> 31)           # 32 -> 34 bits, 6-bit groups are slices of t
        f = (s0[((t >> 28) & 63) ^ g[0]] ^ s1[((t >> 24) & 63) ^ g[1]] ^ s2[((t >> 20) & 63) ^ g[2]] ^
             s3[((t >> 16) & 63) ^ g[3]] ^ s4[((t >> 12) & 63) ^ g[4]] ^ s5[((t >> 8) & 63) ^ g[5]] ^
             s6[((t >> 4) & 63) ^ g[6]] ^ s7[(t & 63) ^ g[7]])
        l, r = r, l ^ f
    return _perm64((r << 32) | l, FPT)

def ecb_v1(data, key8, decrypt=False):
    sks = prepare_keys(key8)
    sks = sks[::-1] if decrypt else sks
    n = len(data) // 8
    words = struct.unpack(">%dQ" % n, data)
    return struct.pack(">%dQ" % n, *[block_v1(w, sks) for w in words])

# ------------------------------ v2 : NumPy, many blocks at once ------------------------------
U = np.uint64
SPN = [np.array(t, dtype=np.uint64) for t in SP]
IPN = [np.array(t, dtype=np.uint64) for t in IPT]
FPN = [np.array(t, dtype=np.uint64) for t in FPT]

def _perm_np(x, T):
    out = T[0][x >> U(56)]
    for j in range(1, 8):
        out |= T[j][(x >> U(56 - 8 * j)) & U(255)]
    return out

def blocks_np(x, sks):
    """x: uint64 array of 64-bit blocks; sks: list of 16 round keys (8 groups each)."""
    x = _perm_np(x, IPN)
    l, r = x >> U(32), x & U(M32)
    for g in sks:
        t = ((r & U(1)) << U(33)) | (r << U(1)) | (r >> U(31))
        f = SPN[0][((t >> U(28)) & U(63)) ^ U(g[0])]
        for i in range(1, 8):
            f ^= SPN[i][((t >> U(28 - 4 * i)) & U(63)) ^ U(g[i])]
```

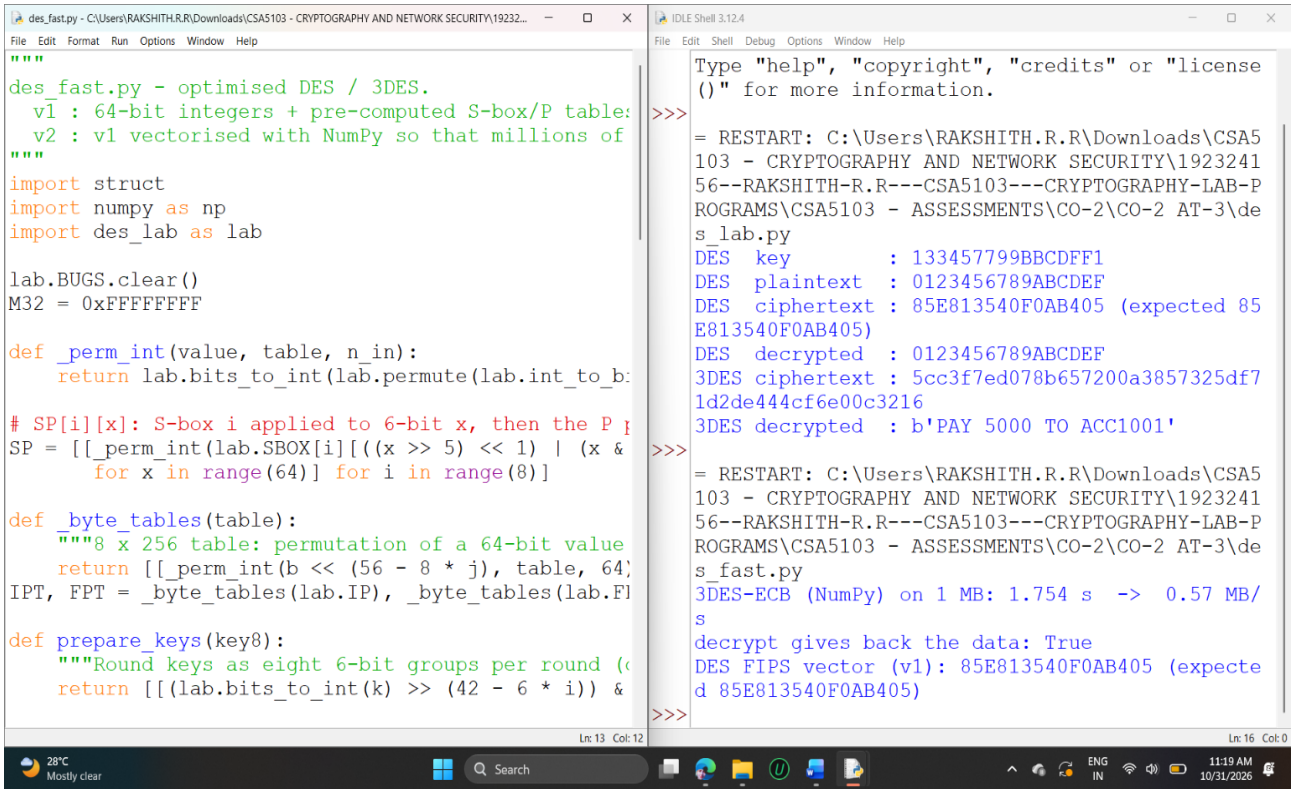
```
        l, r = r, l ^ f
    return _perm_np((r << U(32)) | l, FPN)

def tdes_blocks_np(x, k1, k2, k3, decrypt=False):
    s1, s2, s3 = prepare_keys(k1), prepare_keys(k2), prepare_keys(k3)
    if not decrypt:
        return blocks_np(blocks_np(blocks_np(x, s1), s2[::-1]), s3)
    return blocks_np(blocks_np(blocks_np(x, s3[::-1]), s2), s1[::-1])

def tdes_ecb_np(data, key24, decrypt=False):
    x = np.frombuffer(data, dtype=">u8").astype(np.uint64)
    y = tdes_blocks_np(x, key24[:8], key24[8:16], key24[16:], decrypt)
    return y.astype(">u8").tobytes()

def tdes_ctr_np(data, key24, init_counter=0):
    """CTR mode: keystream = E(counter); encrypt and decrypt are the same operation."""
    nblk = (len(data) + 7) // 8
    ctr = (np.arange(nblk, dtype=np.uint64) + U(init_counter))
    ks = tdes_blocks_np(ctr, key24[:8], key24[8:16], key24[16:]).astype(">u8").tobytes()
    a = np.frombuffer(data, dtype=np.uint8)
    return (a ^ np.frombuffer(ks, dtype=np.uint8)[:len(a)]).tobytes()

def des_ecb_np(data, key8, decrypt=False):
    x = np.frombuffer(data, dtype=">u8").astype(np.uint64)
    sks = prepare_keys(key8)
    return blocks_np(x, sks[::-1] if decrypt else sks).astype(">u8").tobytes()
```



Appendix C – ecc_lab.py (corrected and optimised ECC)

```
"""
ecc_lab.py - elliptic-curve arithmetic over a prime field  y^2 = x^3 + a*x + b  (mod p).

Contains the correct code plus switchable seeded bugs E1 ... E7 (active when the name is in BUGS),
and three scalar-multiplication algorithms: naive repeated addition, double-and-add (affine),
double-and-add in Jacobian coordinates and a Montgomery ladder.
"""
from dataclasses import dataclass
import secrets

BUGS = set()
INF = None                          # the point at infinity (group identity)

@dataclass(frozen=True)
class Curve:
    name: str
    p: int
    a: int
    b: int
    G: tuple
    n: int                          # order of G
```

```
TOY = Curve("toy y^2=x^3+2x+2 mod 17", 17, 2, 2, (5, 1), 19)
P256 = Curve("NIST P-256",
0xFFFFFFFF00000001000000000000000000000000FFFFFFFFFFFFFFFFFFFFFFFF,
        0xFFFFFFFF00000001000000000000000000000000FFFFFFFFFFFFFFFFFFFFFFFC - 3,
        0x5AC635D8AA3A93E7B3EBBD55769886BC651D06B0CC53B0F63BCE3C3E27D2604B,
        (0x6B17D1F2E12C4247F8BCE6E563A440F277037D812DEB33A0F4A13945D898C296,
         0x4FE342E2FE1A7F9B8EE7EB4A7C0F9E162BCE33576B315ECECBB6406837BF51F5),
        0xFFFFFFFF00000000FFFFFFFFFFFFFFFFBCE6FAADA7179E84F3B9CAC2FC632551)
K256 = Curve("secp256k1", 2**256 - 2**32 - 977, 0, 7,
        (0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798,
         0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8),
        0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141)

# ----------------------------------------------------------------- affine arithmetic
def is_on_curve(C, P):
    if P is INF:
        return True
    x, y = P
    return (y * y - (x * x * x + C.a * x + C.b)) % C.p == 0

def divmod_p(num, den, p):
    """num / den in the field GF(p)."""
    if "E5" in BUGS:                    # BUG E5: integer (floor) division, not a modular inverse
        return (num // den) % p
    return num * pow(den, -1, p) % p            # FIX: multiply by the modular inverse

def neg(C, P):
    return INF if P is INF else (P[0], (-P[1]) % C.p)

def point_add(C, P, Q):
    """P + Q on the curve (affine coordinates)."""
    p = C.p
    if "E1" not in BUGS:                # BUG E1: identity element not handled (None unpack -> crash)
        if P is INF: return Q
        if Q is INF: return P
    x1, y1 = P
    x2, y2 = Q
```

```
    if x1 == x2 and (y1 + y2) % p == 0 and "E4" not in BUGS:   # BUG E4: P + (-P) must give the identity
        return INF
    if P == Q and "E3" not in BUGS:                            # BUG E3: doubling case not separated from the chord case
        a_term = 0 if "E2" in BUGS else C.a                    # BUG E2: curve parameter a forgotten in the tangent slope
        lam = divmod_p(3 * x1 * x1 + a_term, 2 * y1, p)
    else:
        lam = divmod_p(y2 - y1, x2 - x1, p)
    x3 = (lam * lam - x1 - x2) % p
    y3 = (lam * (x1 - x3) - y1) % p
    return (x3, y3)

# ---------------------------------------------------------------- scalar multiplication
def mul_naive(C, k, P):
    """k*P by repeated addition: O(k) group operations (only usable for tiny k)."""
    if "E6" in BUGS:                                           # BUG E6: starts at P and loops k times -> (k+1)P, no k mod n
        if k > 100000:
            raise TimeoutError("naive loop would need %d additions" % k)
        R = P
        for _ in range(k):
            R = point_add(C, R, P)
        return R
    k %= C.n
    R = INF
    for _ in range(k):
        R = point_add(C, R, P)
    return R

def mul_double_and_add(C, k, P):
    """Right-to-left double-and-add: O(log k) group operations, affine coordinates."""
    if "E6" in BUGS:
        return mul_naive(C, k, P)
    k %= C.n
    R, Q = INF, P
    while k:
        if k & 1:
            R = point_add(C, R, Q)
```

```
        Q = point_add(C, Q, Q)
        k >>= 1
    return R

# ---- Jacobian coordinates: (X, Y, Z) represents the affine point (X/Z^2, Y/Z^3); no inversion per step
def to_jac(P):
    return (0, 1, 0) if P is INF else (P[0], P[1], 1)

def from_jac(C, J):
    X, Y, Z = J
    if Z == 0:
        return INF
    zi = pow(Z, -1, C.p)
    return (X * zi * zi % C.p, Y * zi * zi * zi % C.p)

def jac_double(C, J):
    p = C.p
    X, Y, Z = J
    if Z == 0 or Y == 0:
        return (0, 1, 0)
    YY = Y * Y % p
    S = 4 * X * YY % p
    M = (3 * X * X + C.a * pow(Z, 4, p)) % p
    X3 = (M * M - 2 * S) % p
    Y3 = (M * (S - X3) - 8 * YY * YY) % p
    return (X3, Y3, 2 * Y * Z % p)

def jac_add(C, J1, J2):
    p = C.p
    X1, Y1, Z1 = J1
    X2, Y2, Z2 = J2
    if Z1 == 0: return J2
    if Z2 == 0: return J1
    Z1Z1, Z2Z2 = Z1 * Z1 % p, Z2 * Z2 % p
    U1, U2 = X1 * Z2Z2 % p, X2 * Z1Z1 % p
    S1, S2 = Y1 * Z2 * Z2Z2 % p, Y2 * Z1 * Z1Z1 % p
    if U1 == U2:
        return jac_double(C, J1) if S1 == S2 else (0, 1, 0)
```

```
    H, R = (U2 - U1) % p, (S2 - S1) % p
    HH = H * H % p
    HHH = H * HH % p
    V = U1 * HH % p
    X3 = (R * R - HHH - 2 * V) % p
    Y3 = (R * (V - X3) - S1 * HHH) % p
    return (X3, Y3, H * Z1 * Z2 % p)

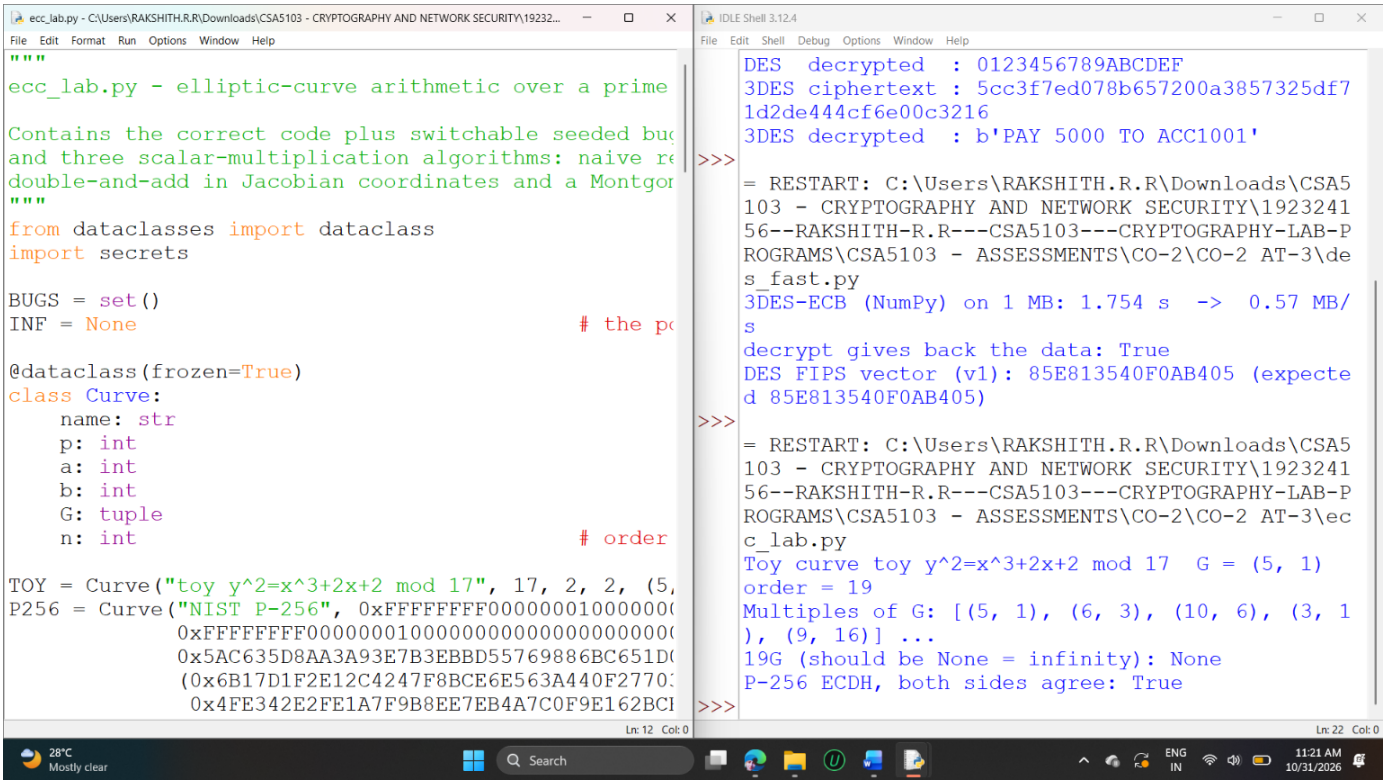
def mul_jacobian(C, k, P):
    """Double-and-add in Jacobian coordinates: ONE modular inversion in total."""
    k %= C.n
    R, Q = (0, 1, 0), to_jac(P)
    while k:
        if k & 1:
            R = jac_add(C, R, Q)
        Q = jac_double(C, Q)
        k >>= 1
    return from_jac(C, R)

def mul_ladder(C, k, P):
    """Montgomery ladder: same sequence of operations for every key bit (side-channel friendlier)."""
    k %= C.n
    R0, R1 = (0, 1, 0), to_jac(P)
    for i in range(max(k.bit_length(), 1) - 1, -1, -1):
        if (k >> i) & 1:
            R0, R1 = jac_add(C, R0, R1), jac_double(C, R1)
        else:
            R1, R0 = jac_add(C, R0, R1), jac_double(C, R0)
    return from_jac(C, R0)

# ---------------------------------------------------------------------- ECDH key exchange
def keygen(C):
    d = secrets.randbelow(C.n - 1) + 1
    return d, mul_jacobian(C, d, C.G)

def ecdh(C, d, Q, mul=None):
    """Shared secret = x-coordinate of d*Q.  The peer's point is validated first (invalid-curve defence)."""
    mul = mul or mul_jacobian
```

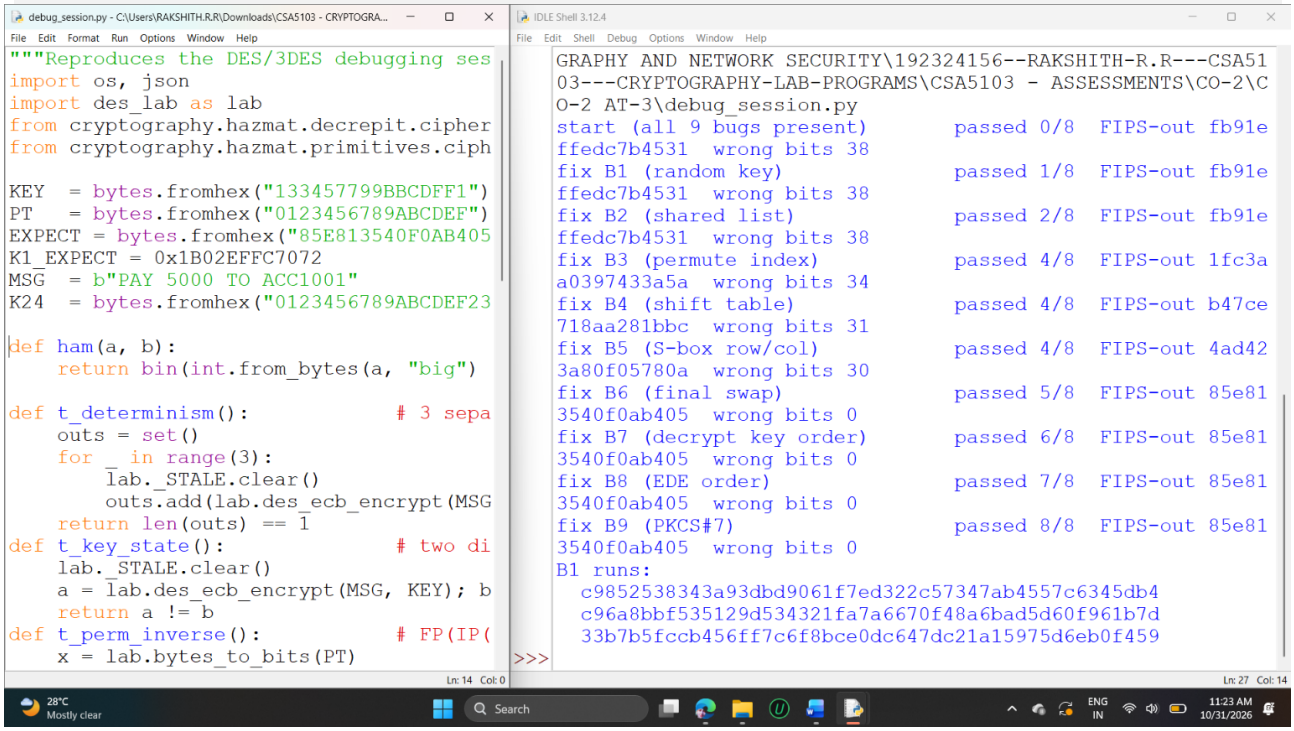
```
if "E7" not in BUGS:                    # BUG E7: peer public key never validated
    if Q is INF or not is_on_curve(C, Q):
        raise ValueError("peer public key is not a valid point on the curve")
S = mul(C, d, Q)
if S is INF:
    raise ValueError("shared point is the identity")
return S[0]
```



Appendix D – Program Output

D1 DES / 3DES debugging session (debug_session.py)

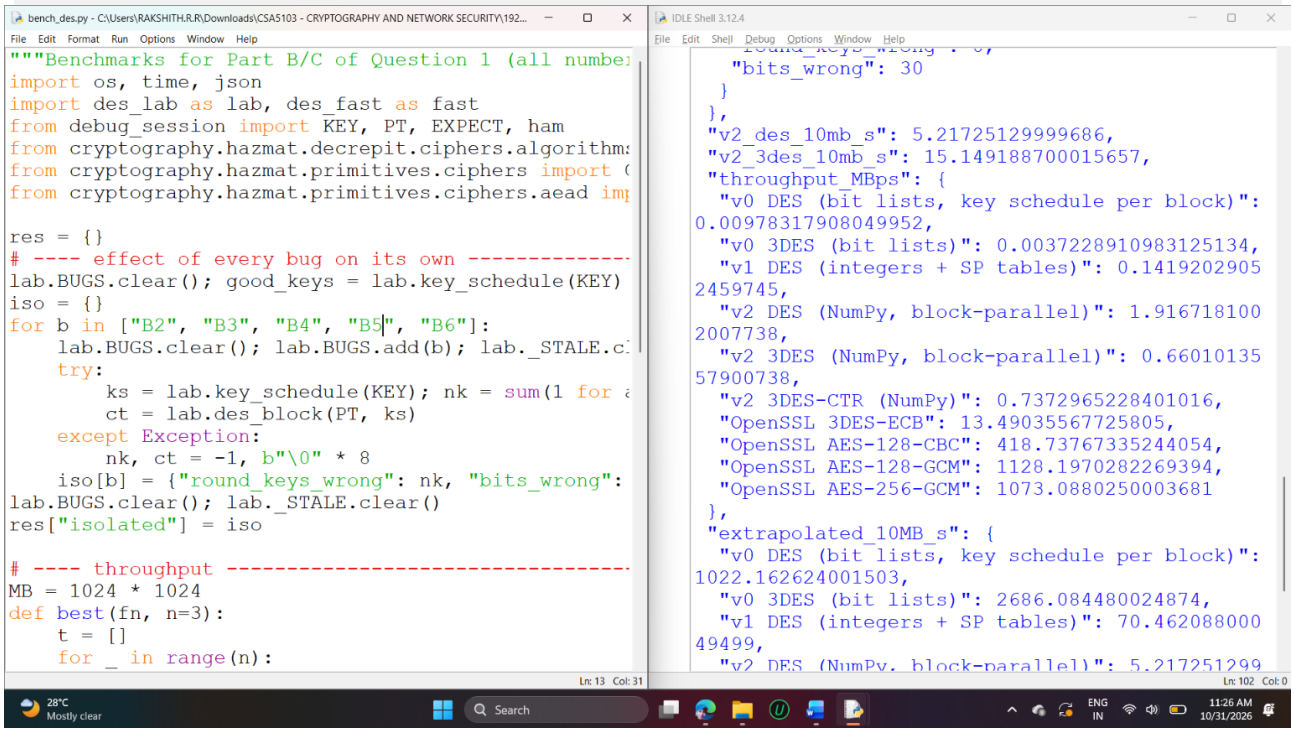
```
start (all 9 bugs present)      passed 0/8  FIPS-out fb91effedc7b4531  wrong bits 38
fix B1 (random key)             passed 1/8  FIPS-out fb91effedc7b4531  wrong bits 38
fix B2 (shared list)            passed 2/8  FIPS-out fb91effedc7b4531  wrong bits 38
fix B3 (permute index)          passed 4/8  FIPS-out 1fc3aa0397433a5a  wrong bits 34
fix B4 (shift table)            passed 4/8  FIPS-out b47ce718aa281bbc  wrong bits 31
fix B5 (S-box row/col)          passed 4/8  FIPS-out 4ad423a80f05780a  wrong bits 30
fix B6 (final swap)             passed 5/8  FIPS-out 85e813540f0ab405  wrong bits 0
fix B7 (decrypt key order)      passed 6/8  FIPS-out 85e813540f0ab405  wrong bits 0
fix B8 (EDE order)              passed 7/8  FIPS-out 85e813540f0ab405  wrong bits 0
fix B9 (PKCS#7)                 passed 8/8  FIPS-out 85e813540f0ab405  wrong bits 0
B1 runs:
  1c81f90f337d735e1c9d38a62069582b809bb886f93c6b9a
  aabb7de066ec952afd7c1a1de8addf809a8907a5ba2c6caf
  fdd9e325a8110bafc0bb04349f11746820e982c856d47d2c
```



Benchmark summary (bench_des.py, 10 MB random data; MB/s):

```
v0 DES (bit lists, key schedule per block)          0.031
v0 3DES (bit lists)                                 0.012
v1 DES (integers + SP tables)                       0.460
v2 DES (NumPy, block-parallel)                      7.838
v2 3DES (NumPy, block-parallel)                     2.497
```

v2 3DES-CTR (NumPy)	2.559
OpenSSL 3DES-ECB	23.578
OpenSSL AES-128-CBC	584.320
OpenSSL AES-128-GCM	6,085.368
OpenSSL AES-256-GCM	6,160.906

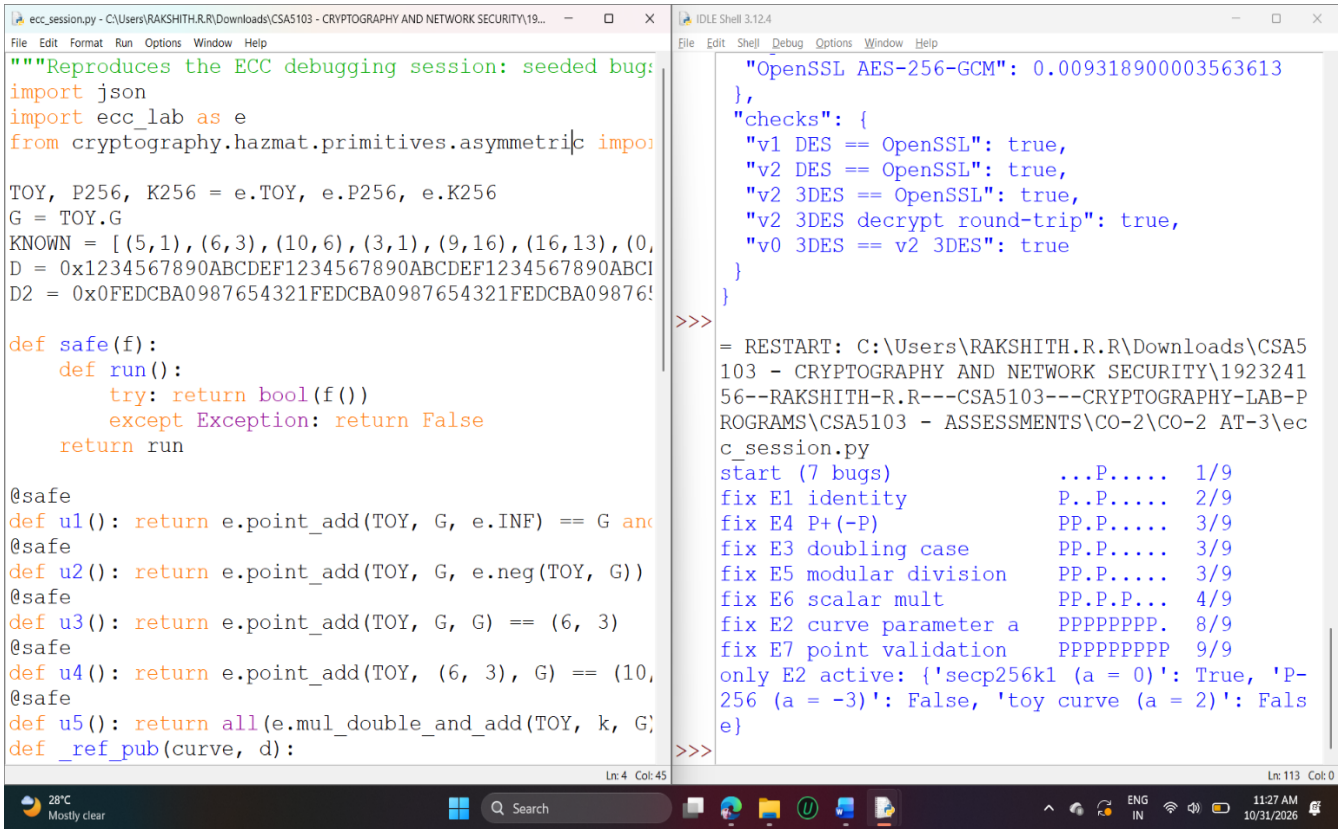


Correctness checks of the optimised code: v1 DES == OpenSSL = true; v2 DES == OpenSSL = true; v2 3DES == OpenSSL = true; v2 3DES decrypt round-trip = true; v0 3DES == v2 3DES = true.

D2 ECC debugging session (ecc_session.py)

start (7 bugs)	...P..... 1/9
fix E1 identity	P..P..... 2/9
fix E4 P+(-P)	PP.P..... 3/9
fix E3 doubling case	PP.P..... 3/9
fix E5 modular division	PP.P..... 3/9
fix E6 scalar mult	PP.P.P... 4/9
fix E2 curve parameter a	PPPPPPPP. 8/9
fix E7 point validation	PPPPPPPPP 9/9

only E2 active: {'secp256k1 (a = 0)': True, 'P-256 (a = -3)': False, 'toy curve (a = 2)': False}



D3 ECC worked examples and ECDH (ecc_demo.py)

== 2G by hand-formula ==========

correct : lambda = (3*5^2+2)/(2*1) = 77/2 = 13 (mod 17) -> 2G = (6,3); on curve: True

bug E2 : lambda = (3*5^2)/2 = 75/2 = 12 (mod 17) -> (15,15); on curve: False

bug E5 : lambda = 77 // 2 = 38 -> reduced mod 17 = 4 (correct value is 13)

== double-and-add trace for 13G ==========

bit 0: 1 add R = (5, 1) Q(= 2^0 G) = (5, 1)

bit 1: 0 skip R = (5, 1) Q(= 2^1 G) = (6, 3)

bit 2: 1 add R = (9, 16) Q(= 2^2 G) = (3, 1)

bit 3: 1 add R = (16, 4) Q(= 2^3 G) = (13, 7)

13G = (16, 4) | table value (16, 4): True

operations: naive repeated addition = 12 additions; double-and-add = 3 useful doublings + 3 additions

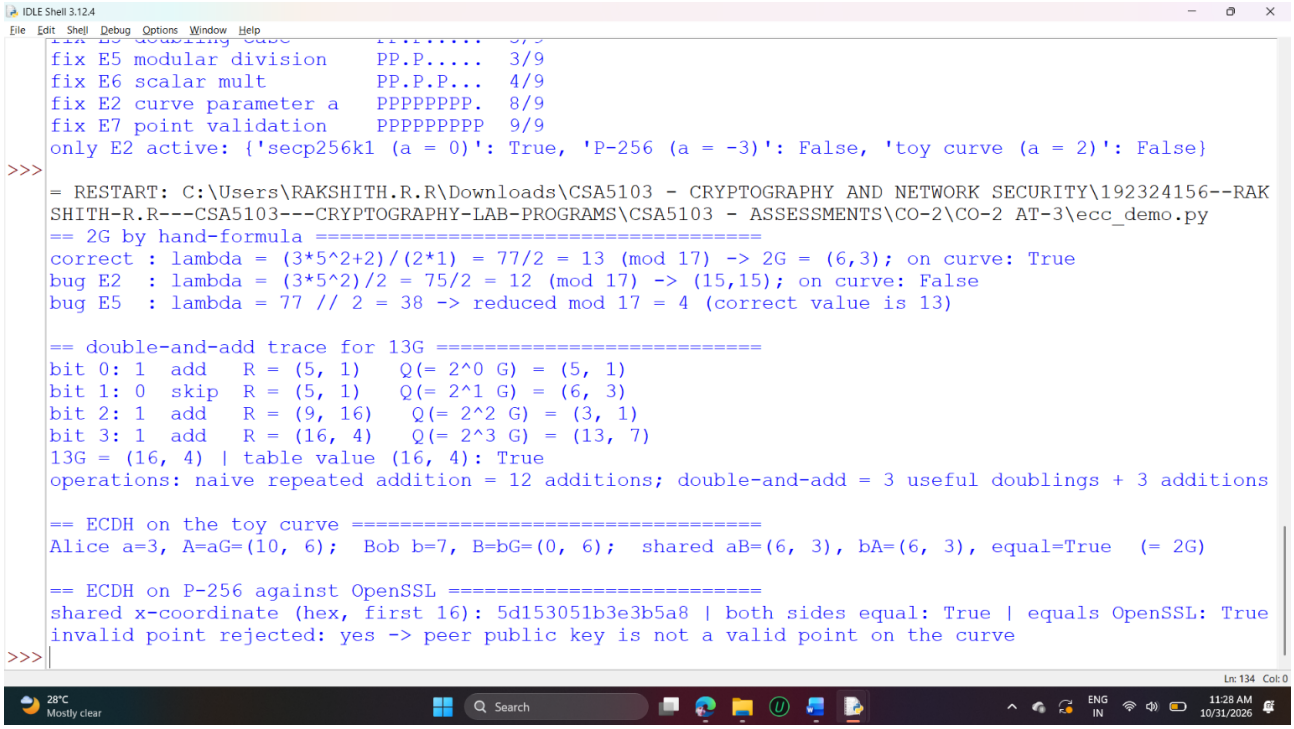
== ECDH on the toy curve ==========

Alice a=3, A=aG=(10, 6); Bob b=7, B=bG=(0, 6); shared aB=(6, 3), bA=(6, 3), equal=True (= 2G)

== ECDH on P-256 against OpenSSL ==========

shared x-coordinate (hex, first 16): 9c49a2d8eb26b4be | both sides equal: True | equals OpenSSL: True

invalid point rejected: yes -> peer public key is not a valid point on the curve



ECC timing summary (ecc_bench.py): Double-and-add, affine = 8.009 ms; Double-and-add, Jacobian = 2.184 ms; Montgomery ladder, Jacobian = 3.118 ms; OpenSSL (C code) = 0.036 ms; pure-Python RSA-2048: RSA-2048 private op, square-and-multiply = 36.96 ms; RSA-2048 private op, with CRT = 10.82 ms; ECC P-256 scalar mult, Jacobian = 2.18 ms.

Environment: Python 3.12 with the cryptography library and NumPy. All timings are from one machine and show relative cost; absolute times on other hardware will differ.

